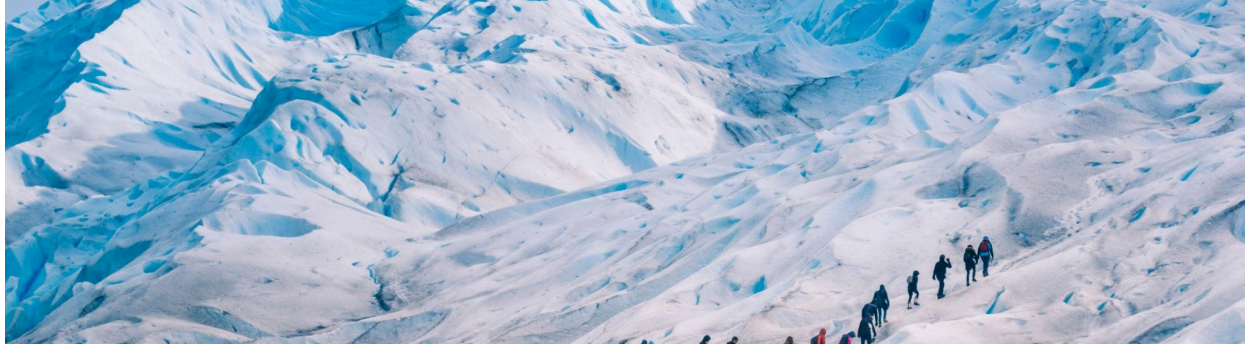




Speedrun

A Desktop and Mobile Study App Built on Anki for One Graduate-Level Exam

Fork Anki. Pick one exam. Build a desktop app and a phone companion that share one engine, measure three different things, change the engine for real, and back up what you show.

1. The Mission

You are not building another flashcard app. Flashcards are great at one thing: helping people remember facts. But a big exam asks for more than memory. Students have to use what they know on new questions they have never seen, work fast enough to finish on time, and figure out if they are actually ready. They also study in two places: at a desk and on their phone between classes. Your tool has to live in both.

Your app has to answer three different questions. They are not the same question:

1. **Memory:** can the student remember this fact right now?
2. **Performance:** can the student answer a new, exam-style question that uses this fact?
3. **Readiness:** what score would the student get today, and how sure are you?

Here is the trap. Remembering the flashcard "the mitochondria is the powerhouse of the cell" does not mean the student can answer an MCAT passage about cellular respiration. Anki's built-in algorithm (FSRS) already estimates memory well. Your job is to build the two harder bridges: from memory to answering new questions, and from answering questions to a real exam score. Measure that gap. Do not hide it.

The honesty rule. Your app may not show a readiness score unless it can also show:

- what evidence produced the number,
- what data is still missing,
- how accurate your past guesses turned out to be,
- the range of likely scores, not just one number,

- and the single best next thing to study.

A confident number with none of that behind it is not a prediction. It is a guess in a nice font.

2. The Rules You Cannot Break

All of these are required. Details come later, but here is the short list:

- Make a real change inside Anki's Rust code, not just the Python screens.
- **Ship two apps that share one engine:** a desktop app and a phone companion. Reviews and progress sync between them.
- Show three separate scores (memory, performance, readiness), each with a range, not one blended number.
- Test your models on data you held back, using a setup someone else can re-run and get the same result.
- Pick one study feature, write down what you expect it to do, and test it by turning it off and on.
- Every AI output must come from a named source, get checked against a test set, and beat a simpler method.
- The app must refuse to give a score when it does not have enough data.
- Ship a desktop installer and a phone build that both run with AI switched off.
- Follow the license: AGPL-3.0-or-later, with credit to Anki. (Some parts of Anki use BSD-3-Clause.)

Making up a readiness number, or dressing up a guess as a measurement, is an automatic fail. We trust honest numbers over flattering ones.

3. Two Apps, One Engine

The desktop app is the main tool. The phone app is a companion for reviewing on the go and checking readiness. They are not two separate projects. They share the same cards, the same progress, and the same engine, and they sync.

What the phone companion must do:

- Run real review sessions on the same deck as the desktop app.
- Sync both ways with the desktop: review on the phone, see it on the desktop, and the reverse.
- Work offline, then sync when the connection comes back.
- Show the same three scores with ranges, and follow the same give-up rule.

How to build it (share the engine, do not rewrite it):

- **Android:** build on AnkiDroid (open source, AGPL), or run Anki's Rust backend on the device.

- **iOS:** run Anki's Rust backend through its C interface (FFI), the same approach real Anki-compatible iOS clients use.

You may use Anki's existing sync, or build your own. Either way, reviews must flow between the two apps without losing or double-counting data. Rewriting the scheduler in JavaScript or Swift instead of sharing the Rust engine does not count.

4. Memory, Performance, Readiness (Show Them Separately)

Mixing these three together is the easiest way to fail this project. Anki's FSRS handles memory. A great memory model is not a great score model. Build and show all three.

- **Memory:** the chance the student recalls a fact you taught.
- **Performance:** the chance the student gets a new, exam-style question right, including ones they have never seen.
- **Readiness:** a projected score on the real scale, with a range and a note on how sure you are.

A good readiness display looks like this, not like a single "78% ready":

Projected MCAT: 508

Likely range: 503 to 512

Confidence: low, because you have only studied 42% of the topics on the exam.

Every score must come with: the point estimate, the likely range, the percent of the exam covered so far, a "how sure" indicator, the time it was last updated, the main reasons behind it, and the rule for when it gives up and shows nothing.

The give-up rule (write yours down). The app must show no score when it lacks data. Set a clear line and state it. Example: "No score until the student has at least 200 graded reviews and 50% topic coverage." A good system knows when it does not know.

5. Pick One Exam

Choose one exam before you write code. Build the whole app for it, on its real scoring scale. Map its sections, scoring, and time limits. State your exam at the top of your README. Build it as it works today:

- **MCAT:** scored 472 to 528, four sections each scored 118 to 132. Huge fact base plus reading passages. The hard part is covering it all.
- **LSAT:** scored 120 to 180. Logic games are gone as of August 2024. The test is now two Logical Reasoning sections and one Reading Comprehension section. Do not build

around logic games unless you are aiming at old exams on purpose. There are almost no facts to memorize, so a flashcard-based score model is genuinely tricky here.

- **GMAT (Focus Edition):** total 205 to 805 in steps of 10. Three sections (Quantitative, Verbal, Data Insights), 45 minutes each, section scores 60 to 90. The test adapts to the student, so model that.
- **USMLE Step 1:** pass/fail only since January 26, 2022. Do not invent a number score. Predict the chance of passing or readiness above a safe line.

6. What Is Due, Deadline by Deadline

Build in order: make the apps work, add AI, then prove it. Do not skip ahead. Each deadline needs proof, not a promise. **No AI before Friday:** the Wednesday build has no model calls, no generated cards, no chatbot.

Deadline	Headline
Wednesday	Both apps work and review the same deck. No AI.
Friday	AI is added and checked. Phone syncs with desktop and shows readiness.
Sunday	You prove it works, and ship installable builds for both.

Due Wednesday: the core works on both screens (no AI)

Desktop:

- Anki forked and building from source.
- Your Rust change working end to end (the diff, 3 Rust unit tests, and 1 test that calls it from Python).
- A review loop running on your exam deck.
- A memory model running, with an honest score: a range plus the give-up rule.
- An installer that runs on a clean machine.

Mobile:

- A phone app that builds and runs on a real device or emulator.
- It loads your exam deck and runs a real review session on the shared engine. (Two-way sync is not required yet; reviewing the same deck is.)

Proof:

- Commit hash and a clean-build recording, the test results, a clean-machine install recording, and a screen recording of a review session on the phone.

Due Friday: AI added and checked; phone syncs

Desktop (AI):

- A short note on what AI you built, why, and what you skipped.
- Every AI output traces back to a named source.
- An eval that runs before students see anything: accuracy and wrong-answer rate on a held-out set, with your cutoff.
- A side-by-side showing your AI beats a simpler method (keyword or vector search).
- The app still gives a score with AI switched off.

Mobile:

- Two-way sync with the desktop works: review on the phone, see it on the desktop, and the reverse, with no lost or double-counted reviews.
- Offline review works, then syncs when the connection returns.
- The phone shows the three scores with ranges and follows the give-up rule.

Proof:

- The eval numbers and baseline comparison, plus a recording of a card reviewed on the phone showing up on the desktop after sync.

Due Sunday: prove it, and ship both

Models and evidence:

- Memory model is calibrated: a calibration chart and a score (Brier or log loss) on held-out reviews.
- Performance model: accuracy on held-out exam-style questions.
- Score mapping: your method written down, with a range.
- Your study feature tested with three builds (see section 8), equal study time.
- Honest reporting, including results that did not work.

Desktop and mobile:

- A packaged desktop installer and a packaged phone build (signed APK, or iOS via TestFlight or a sideload build).
- Sync handles conflicts: if both devices review the same card offline, the merge is correct and documented.
- Both apps run with AI off and still give a score.

Proof:

- The results report, the model descriptions, the Brainlift, and recordings of both builds installing and running on clean devices.

7. Concrete Challenges

These are specific tasks, with numbers, so there is no hiding behind big words. Do the real

version of each.

7a. The Rust change (pick one to start)

"Work inside Anki's existing code" (a brownfield project) means you must change the Rust engine, not just recolor the Python screens. Start with one of these, or something of equal weight:

- **Points-at-stake queue:** add a new review order that sorts due cards by topic weight times student weakness, so the highest-value cards come first. Add a new protobuf message for it and call it from Python.
- **Topic-aware scheduling:** bring weak-topic cards back sooner while keeping FSRS intervals valid and undo working.
- **Mastery query:** add a backend call that returns, per topic, how many cards are mastered and the average recall, fast enough to power the dashboard on 50,000 cards.

Whatever you pick, you must include:

- at least 3 Rust unit tests and 1 test that calls your change from Python,
- proof that undo still works and the collection does not corrupt,
- a one-page note on why this belongs in Rust, not Python,
- a list of the upstream files you touched and how hard a future merge would be.

Because the engine is shared, your Rust change ships to both the desktop and the phone. Make sure it still works on the phone build.

7b. The sync test

Review 10 cards on the phone while offline. Review 10 different cards on the desktop. Reconnect. Show that all 20 reviews land in one place with none lost and none counted twice. Then review the same card on both devices offline, sync, and show your conflict rule picks a clear, correct winner. Write down what that rule is.

7c. The coverage map

List every topic on your exam's official outline. Mark which ones your deck actually covers. Show the percent covered on the dashboard. If coverage is below your line, the app abstains from a score. A deck with 10,000 cards that skips a whole high-weight section should not show "ready."

7d. The paraphrase test

This proves you measure performance, not just memory. Take 30 cards. For each, write 2 exam-style questions that test the same idea in new words. Compare the student's recall on the card with their accuracy on the reworded questions. If the two numbers are basically the same, your performance model is just copying your memory model, and you have not built the bridge. Report the gap.

7e. The leakage check

Write a script that scans your training data and flags any test item, or a near-copy of one, that slipped in. Run it and show the result is clean. Leaked data makes a model look smarter than it is, and it zeroes that score.

7f. The AI card check

Build a gold set: 50 question-and-answer pairs for your exam with known-correct answers. Then generate 50 cards from one real source (a textbook chapter or notes) and run them through your checker. Report three counts:

- how many were correct and useful,
- how many were wrong (a wrong fact is worse than no card),
- how many were correct but bad teaching (vague, trivial, or duplicates).

Set a passing cutoff before you look at the results, and block any card that fails it.

7g. The crash and offline tests

Kill each app in the middle of a review 20 times in a row. Show zero corrupted collections afterward. Then pull the network: AI features should turn off cleanly while both apps keep working and still give a score.

7h. The one-command benchmark

Ship a single command (for example, `make bench`) that loads the shared 50,000-card deck and prints your speed numbers: the median, the 95th percentile, and the worst case for each action below. One hand-picked number does not count.

8. How to Test a Study Feature

Pick one study feature based on learning science. Write down, in one sentence, what you expect it to do and how you will know if it failed. Example: "Mixing related topics in one session (interleaving) will raise accuracy on new mixed-topic questions at the same study time."

Then test it by building three versions and comparing them on the same questions, with the same study time:

1. **Your full app** (feature on).
2. **Your app with that one feature turned off** (this is the "ablation," cutting away one part to see what it was doing).
3. **Plain, unmodified Anki** (the baseline).

Why all three? If your full app beats plain Anki, you still do not know which change helped. It could be your better cards or just more reviews. Turning the one feature off (version 1 vs version 2) shows whether that feature did the work. Comparing to plain Anki (version 3) shows whether your whole app beats the obvious alternative at all.

Your final test must use the same learners, the same questions, and the same time budget for all three. State your main number ahead of time. Report a range, and report results that did not work.

You do not have to prove your idea works. You have to run a fair test that could show it does not. "Interleaving made no difference here" is a real, useful result and scores well. "Our app feels better" scores nothing.

9. What to Build the Score Model On

A true score model needs students studying and then taking real practice tests, tracked over time. You will not honestly gather that in a week. So we grade the steps of the bridge, not a made-up final number:

- **Step 1 (required):** show your memory model is calibrated. When it says 80%, the student should recall about 80% of the time. Prove it on held-back reviews.
- **Step 2 (required):** predict whether the student gets held-back exam-style questions right, using topic mastery, question difficulty, timing, and coverage.
- **Step 3 (required):** turn question performance into a score, explain your method, and show a range.
- **Step 4 (bonus):** check against real students who have both study history and practice-test scores.

Saying "we calibrated memory, but we do not yet have the data to prove the projected score is right" scores higher than a polished score you cannot back up.

10. Speed and Reliability Targets

Measured on the shared deck and reference machines. Report the median (p50), the 95th percentile (p95), and the worst case for each:

- Button press is acknowledged: p95 under 50 ms (desktop and phone).
- Next card appears after grading: p95 under 100 ms.
- Dashboard first load: p95 under 1 second.
- Dashboard refresh: p95 under 500 ms, without freezing the screen.
- Sync of a normal session: completes in under 5 seconds on a normal connection.
- Memory use on 50,000 cards: under a limit you state, on both desktop and a mid-range phone.
- App cold start: under 5 seconds on desktop, under 4 seconds on the phone.
- Nothing freezes the screen for more than 100 ms.
- Zero corrupted collections in the crash test, on both platforms.

We will try to break it. Be ready for:

- a student who memorizes card wording but fails reworded questions,

- a deck that is huge but skips a high-weight topic,
- two cards that state opposite facts,
- a source file with hidden text trying to trick your card generator (prompt injection),
- a student who taps Good without reading,
- a topic with almost no history,
- a student who is accurate but too slow for the time limit,
- AI cards that are correct but useless,
- a score that jumps up only because test questions leaked into training,
- the AI service being offline, rate-limited, or returning broken output,
- the same card reviewed on two devices offline, then synced (your conflict rule must hold),
- a phone that goes offline mid-sync, or whose clock is set wrong,
- a crash in the middle of a review that must recover cleanly,
- a corrupt deck, a 50,000-card deck, and a deck with broken images.

11. How We Grade

Area	Weight
Rust change and how well it fits Anki	20%
Score accuracy and honest uncertainty	20%
Study feature built on learning science	15%
AI checking and safety	15%
Fair tests others can re-run	12%
Desktop and phone sharing one engine, with working sync	10%
Useful product and clean UX (both apps)	8%

Hard limits:

- No real Rust change: 50% maximum.
- No phone companion that shares the engine and syncs: 70% maximum.
- No re-runnable test setup: 60% maximum.
- No held-out testing: 60% maximum.
- Made-up or misleading readiness numbers: automatic fail.
- Either app does not run on a clean device: 50% maximum.
- Leaked test data: that score is zero.

- AI claims with no traceable source: the AI section is zero.

12. What to Hand In

Due **Sunday 10:59 PM CT**:

- **GitHub repo**: a public AGPL-3.0-or-later fork with credit to Anki, your exam stated up front, build instructions for both apps, an architecture overview, the note on your Rust change, and the list of files you touched.
- **Demo video (3 to 5 minutes)**: a review session, your Rust change in action, a card synced from phone to desktop, the three scores with ranges, your AI features, and your test results.
- **Model descriptions**: one short page each for the memory, performance, and readiness models, including the give-up rule.
- **Brainlift**: As per Patrick's descriptions and class outline.

13. Feature Ideas (Only If the Core Is Solid)

- Real-time sync: a review on one device shows on the other within a second, no manual sync.
- End-to-end encrypted sync, or a conflict-free merge that never needs a winner picked.
- Hit 100,000 cards on both platforms while staying inside your speed targets, and show the profiling.
- Ship signed, notarized installers for macOS, Windows, and Linux, plus a store-ready phone build.
- Get a real change accepted into Anki or AnkiDroid upstream, or publish a useful add-on.
- For a reasoning exam, model LSAT Logical Reasoning and Reading Comprehension questions, or model GMAT's adaptive scoring.
- Use a knowledge graph for study planning, and prove it beats plain keyword and vector search with real numbers. The graph alone is not the win. Showing it helps is the win.

Get Anki Building First

The hardest part of day one is not your features. It is getting Anki to compile from source, making one tiny Rust change show up in the desktop app, and getting the same engine running on a phone. Do that before anything else. Teams that leave the Rust build or the mobile build until Thursday will not finish.

One exam, two apps on one engine, a real engine change, and three scores you can back up beats a flashy app that promises everything and proves nothing.

Build something real, then try to prove yourself wrong.